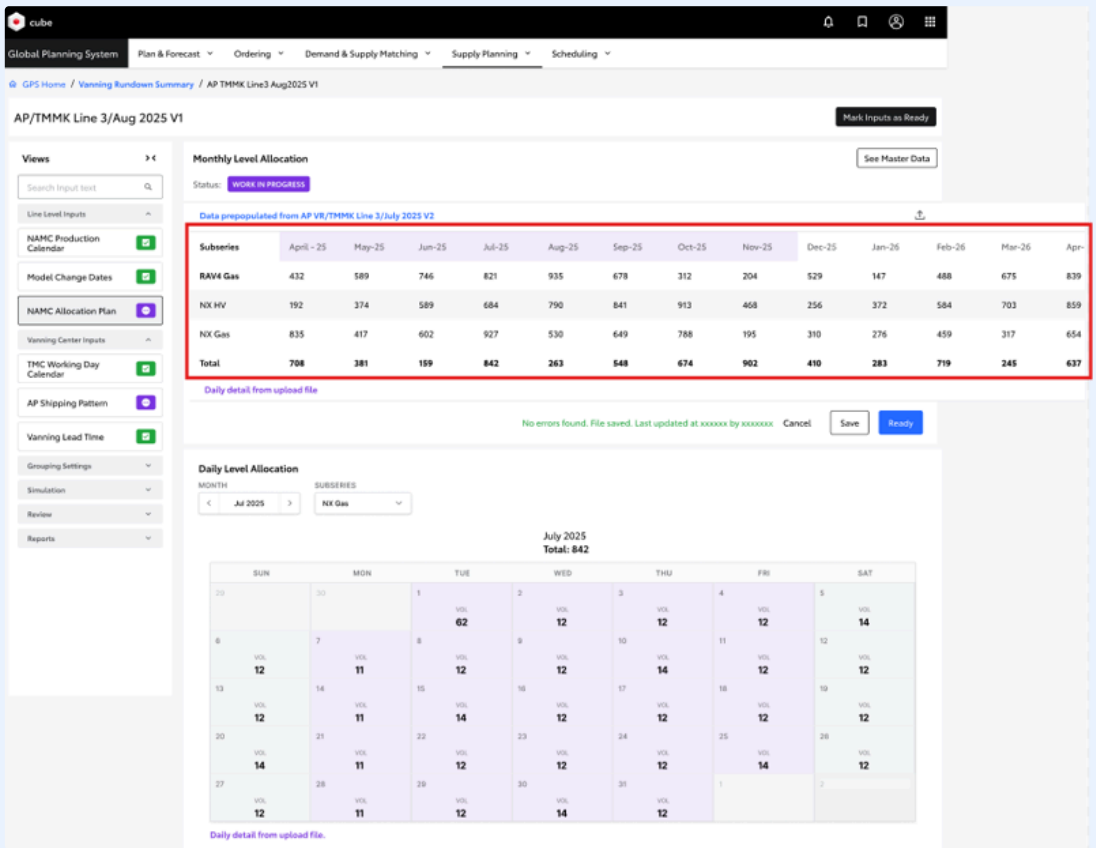
SPPL-373: NAMC Allocation Plan Post API technical workflow

Jira Ticket: SPPL-373: BE | Create technical workflow for NAMC Allocation Plan Post (including redistribution logic) API on parameter inputs page including DB details ACCEPTED

This page provides comprehensive information about the POST API for Monthly NAMC Allocation Plan on the Input params page, to update NAMC allocation plan at monthly level.



Supply Planning POST NAMC Allocation Plan API Details:

1. **Hosted URL**: https://api.spln.cube<env>.toyota.com/<env>
2. **API Name:** <HOSTED_URL>/v/1/1/sp/monthly-namc-allocation-plan
3. **Path Param**: N/A
4. **Query Param**: N/A
5. **Method:** POST
6. **Authorization**: Bearer token (**required**)
7. **Lambda Name**: spln-<env>-update-monthly-namc-allocation-plan-v1-1
8. **API Request Body**:

```
{
  "scenarioId": "uniqueScenarioId",
  "userEmail": "user@toyota.com"
  "data": [
    {
      "monthYear": "Jan-25",
      "subSeries": "RAV4 Gas",
      "monthlyVolume": 100
    },
    {
      "monthYear": "Apr-25",
      "subSeries": "CAMRY",
      "monthlyVolume": 200
    }
  ]
}
```

9. **API Response:**
   a. **Success response with status 200:**

```
{
  "message": "Successfully updated data."
}
```

   b. **Error Response:**
      - **any internal server error occurred with status 500:**

```
{
  "errorMessage": "Internal Server Error"
}
```

      - **any unauthorized error occurred with status 401 (Managed by the AWS API Gateway integrated Custom Authorizer):**

```
{
  "message": "Unauthorized"
}
```

      - **If request is in-valid with status 400:**

```
{
  "errorMessage": [<"ValidationError: validation error message”>]
}
```

      - **List of possible validation error message:**
        1. "ValidationError: Request body cannot be empty.”
        2. "ValidationError: scenarioId is required and must be a uuid."
        3. "ValidationError: userEmail is required and must be a string."
        4. "ValidationError: Invalid userEmail."
        5. "ValidationError: Scenario doesn't exist."
        6. "ValidationError: data is required and must be an array with atleast 1 item."

7. "ValidationError: monthYear is required and must be a string in the format MMM-YY."
8. "ValidationError: subSeries is required and must be a string."
9. "ValidationError: monthlyVolume is required and must be a non-negative number not more than 99,999."
10. “ValidationError: monthYear & subSeries combination must be unique.“

10. **Table:**
    a. **scenarios**
    b. **namc_allocation_plan**
    c. **scenario_step_status**

SP Post Monthly NAMC Allocation Plan API Workflow:

- **Technical workflow:**
  - Description:
    i. The Supply Planning user logs in to the application.
    ii. After the user provides creates a scenario, opens it and updates monthly NAMC Allocation Plan data, a request is sent to the SP Post Monthly NAMC Allocation Plan endpoint: ***sp/monthly-namc-allocation-plan*** with payload:

```json
{
  "scenarioId": "uniqueScenarioId",
  "userEmail": "user@toyota.com"
  "data": [
    {
      "monthYear": "Jan-25",
      "subSeries": "RAV4 Gas",
      "monthlyVolume": 100
    },
    {
      "monthYear": "Apr-25",
      "subSeries": "CAMRY",
      "monthlyVolume": 200
    }
  ]
}
```

    iii. After successful authentication, the **spln-update-monthly-namc-allocation-plan-v1-1** lambda is triggered by the SP update scenario Monthly NAMC Allocation Plan endpoint: ***sp/*monthly-*namc*-allocation-plan**. If authentication fails, an error with status 401 (*refer: error response-**any unauthorized error occurred with status 401**)* will be returned and sent to client.
    iv. After user authentication has been confirmed, the **spln-update-monthly-namc-allocation-plan-v1-1** lambda will perform validation on the request payload. If

payload is valid will proceed to next step else throw validation error (*refer: error response*: ***Section if request is in-valid with status 400 and List of possible validation error message***). Below query is used to check if provided scenario is valid:

```
select * from supply_planning.scenarios where scenario_id = :scenarioId and
is_active = true;
```

v. After the request payload is successfully validated, the **spln-update-monthly-namc-allocation-plan-v1-1** lambda execute below logic to calculate daily level redistribution based on NAMC Production Calendar Data. Use below queries to fetch NAMC Production Calendar Data:

```
SELECT * FROM supply_planning.namc_production_calendar where scenario_id =
<inputScenarioId> and production_month in (<inputMonthYear>);
```

1. Check if we have NAMC Production Calendar data for the edited monthYear (refer above select query (v) response),
   a. If yes - redistribute the monthly allocation value for daily level based on the NAMC Production Calendar data
      i. Best way to implement redistribution logic - For example,
         a. I have a monthly value, such as 200 for Apr 2025, which needs to be allocated across each working day in the month (with non-working days receiving 0).
         b. First, retrieve all dates for that month from the NAMC Production Calendar table (refer to above query (v) response), for the month year—let's say there are 30 dates, with 10 being non-working.
         c. Next, distribute the 200 across the working days using the percentage assigned to each working date. These percentages come from the same NAMC Production Calendar table (refer to above query (v) response) for the month, where non-working days have 0% and working days have values between 1 and 200.
   b. If no - skip

vi. Update scenario NAMC Allocation Plan data in the namc_allocation_plan table

```
-- Only monthly volume update (in case of no NAMC Production Calendar Data for
given input)
UPDATE supply_planning.namc_allocation_plan n
    SET
      monthly_volume = v.monthly_volume,
      last_updated_timestamp = NOW(),
      updated_by = ${userEmail}
    FROM (
      VALUES
        ${Prisma.join(
```

```
          data.map((item) => Prisma.sql`(
            ${item.monthYear}::varchar,
            ${item.subSeries}::text,
            ${item.monthlyVolume}::int
          )`)
        )}
    ) AS v(production_month, sub_series_description, monthly_volume)
    WHERE
      n.scenario_id = ${scenarioId}::uuid
      AND n.production_month = v.production_month
      AND n.sub_series_description = v.sub_series_description
      AND n.editable = true;

-- Both daily and monthly volume update
UPDATE supply_planning.namc_allocation_plan n
    SET
      daily_volume = v.daily_volume,
      monthly_volume = v.monthly_volume,
      last_updated_timestamp = NOW(),
      updated_by = ${userEmail}
    FROM (
      VALUES
        ${Prisma.join(
          data.map((item) => Prisma.sql`(
            ${item.prodDate}::date,
            ${item.monthYear}::text,
            ${item.subSeries}::text,
            ${item.dailyVolume}::int,
            ${item.monthlyVolume}::int
          )`)
        )}
    ) AS v(production_date, production_month, sub_series_description, daily_volume, monthly_volume)
    WHERE
      n.scenario_id = ${scenarioId}::uuid
      AND n.production_date = v.production_date
      AND n.production_month = v.production_month
      AND n.sub_series_description = v.sub_series_description
      AND n.editable = true;
```

vii. Update the step & scenario status to “In Progress”. Only update scenario status to “In Progress” if its not already using scenarioData from query (iv) response.

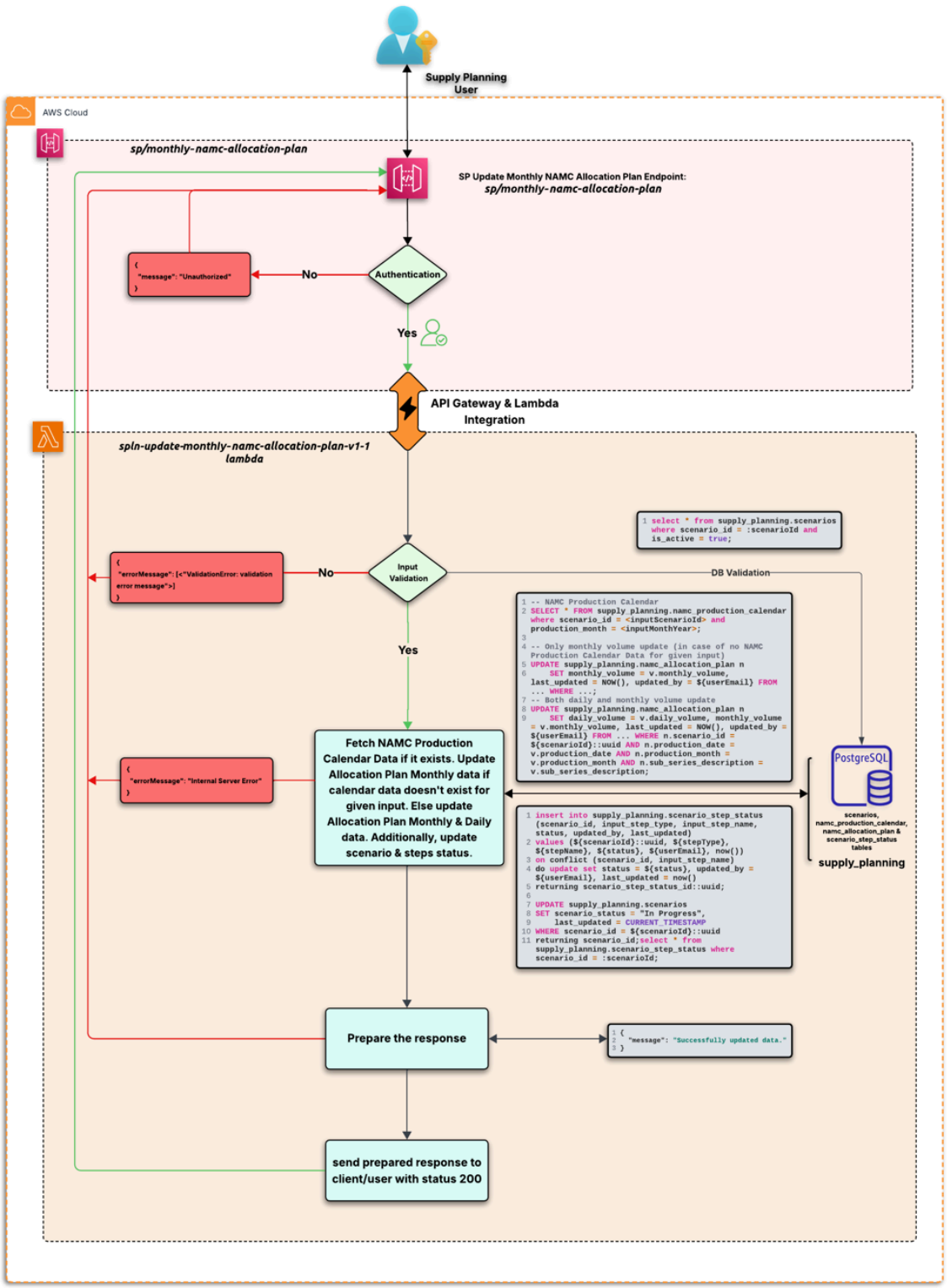
```
insert into supply_planning.scenario_step_status (scenario_id, input_step_type, input_step_name, status, updated_by, last_updated)
values (${scenarioId}::uuid, ${stepType}, ${stepName}, ${status}, ${userEmail}, now())
on conflict (scenario_id, input_step_name)
do update set status = ${status}, updated_by = ${userEmail}, last_updated = now()
returning scenario_step_status_id::uuid;

UPDATE supply_planning.scenarios
SET scenario_status = "In Progress",
    last_updated = CURRENT_TIMESTAMP
WHERE scenario_id = ${scenarioId}::uuid
returning scenario_id;select * from supply_planning.scenario_step_status where scenario_id = :scenarioId;
```

viii. After updating the NAMC Production Calendar data, return the below response to the client with a status code of 200:

```
{
  "message": "Successfully updated data."
```

```
}
```

ix. If there is an error while updating the data, an “Internal Server Error” with a status code of 500(*refer: error response-**any internal server error occurred with status 500***) will be returned. Otherwise, a successful response with a status code of 200 will be sent.



SP Monthly NAMC Allocation Plan Post API Techncial workflow

Note:

We will be utilizing this API for both to fetch initial data and previous rundown data .

Queries: